

Benchmarking Dense and Sparse Matrix Multiplication Algorithms in Java

Nerea Valido Calzada

November 22, 2025

Project Repository: https://github.com/fenfiera/Big_Data-Individual_assignments/tree/main/Individual_Assignment_2

Abstract

Matrix multiplication is a fundamental operation in scientific computing, machine learning, and data analysis. This paper presents a comparative benchmarking study of three matrix multiplication algorithms: Naive, Strassen's optimized method, and a Sparse representation approach implemented in Java and evaluated using the Java Microbenchmark Harness (JMH). Experiments were conducted across varying matrix sizes and sparsity levels to assess performance trade-offs. Results highlight the scalability limitations of naive multiplication, the stability of Strassen's method, and the efficiency of sparse multiplication under high sparsity conditions. AI has been used in this work.

1 Introduction

Matrix multiplication is central to numerous computational tasks, from linear algebra to deep learning. Optimizing this operation is crucial for performance in large-scale applications. Traditional dense multiplication scales poorly with matrix size, while advanced algorithms such as Strassen's method and sparse representations offer potential improvements. This study benchmarks these approaches to provide empirical insights into their performance characteristics.

2 Methodology

2.1 Implementation

- **Naive Multiplication:** Standard triple-loop algorithm with $O(n^3)$ complexity.
- **StrassenOptimized:** Recursive divide-and-conquer algorithm with loop unrolling and cache blocking for small subproblems.
- **SparseMatrix:** HashMap-based storage of non-zero entries, optimized for high sparsity.

2.2 Benchmarking Framework

- **Tool:** Java Microbenchmark Harness (JMH).
- **Parameters:** Matrix sizes $n = 128, 512, 1024$; sparsity levels 0%, 25%, 50%, 75%, 90%.
- **Metrics:** Average execution time (ms/op).
- **Configuration:** 5 warmup iterations, 5 measurement iterations, 5 forks.

3 Results

3.1 Naive Multiplication

Efficient for small matrices ($n = 128$, ~ 1.3 ms/op). Performance degrades rapidly with size ($n = 1024$, $\sim 3\text{--}7$ seconds/op). Sparsity has limited impact due to dense storage.

3.2 Sparse Multiplication

Poor performance on dense matrices ($n = 1024$, ~ 50 seconds/op). Significant speedup with sparsity $\geq 75\%$ ($n = 1024$, ~ 1 second/op). Best suited for highly sparse datasets.

3.3 StrassenOptimized

Stable across sizes: ~ 1.8 ms/op for $n = 128$, ~ 750 ms/op for $n = 1024$. Outperforms naive multiplication consistently. Less sensitive to sparsity due to dense representation.

3.4 Figures

Figures 1, 2, and 3 show the comparative performance of the three algorithms across different sparsity levels.

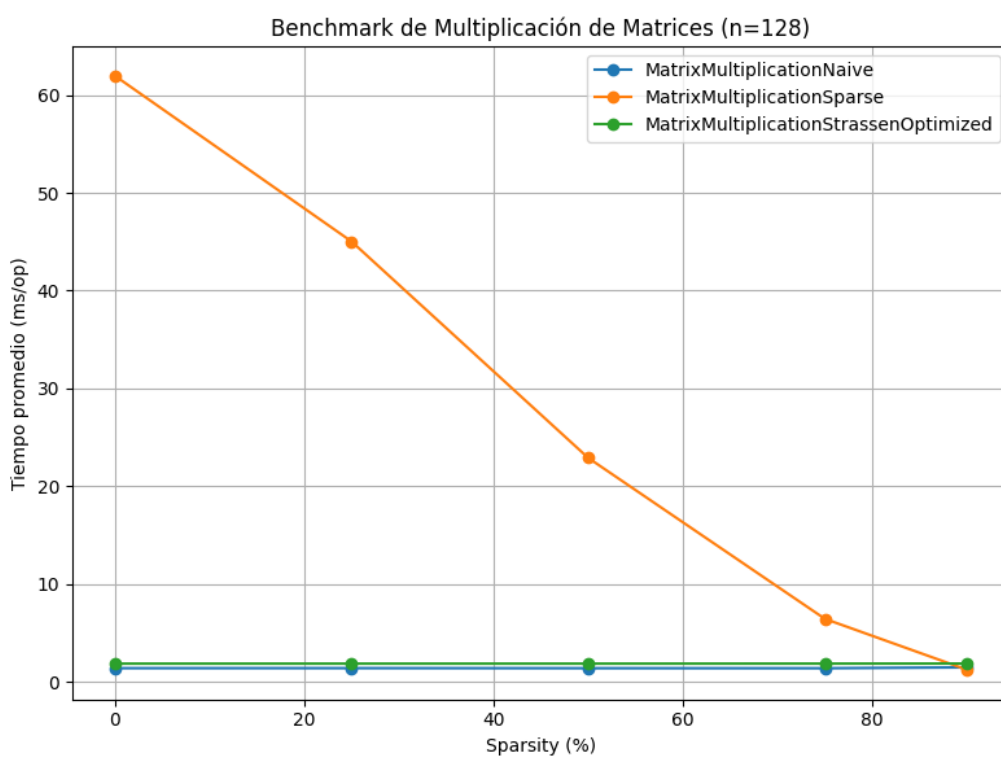


Figure 1: Performance comparison for $n = 128$ across sparsity levels.

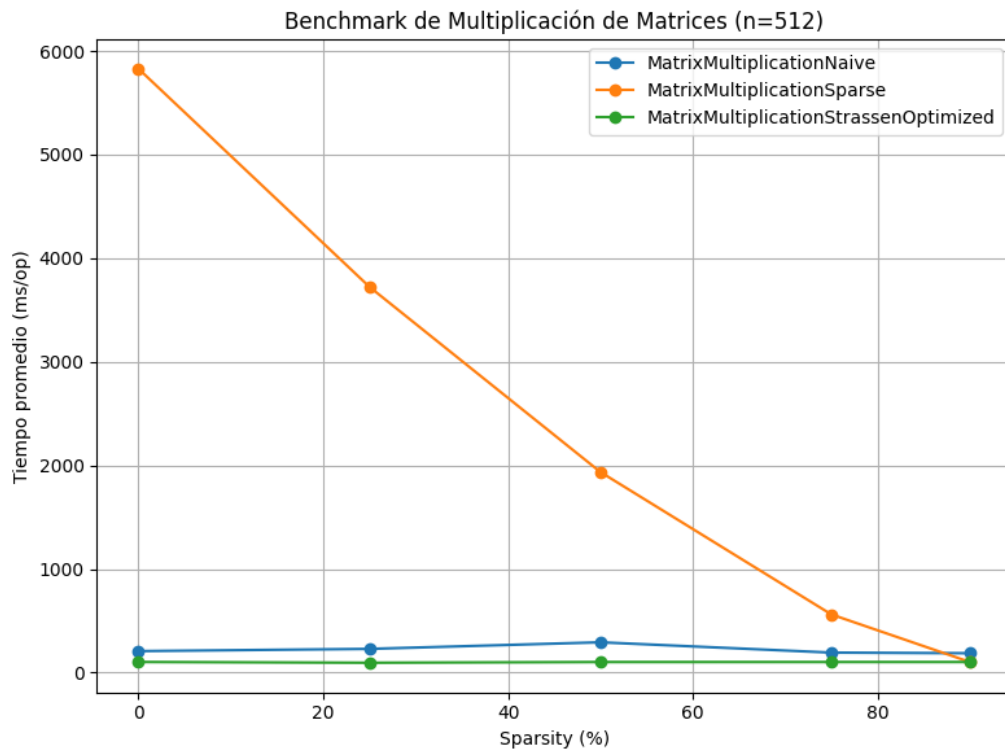


Figure 2: Performance comparison for $n = 512$ across sparsity levels.

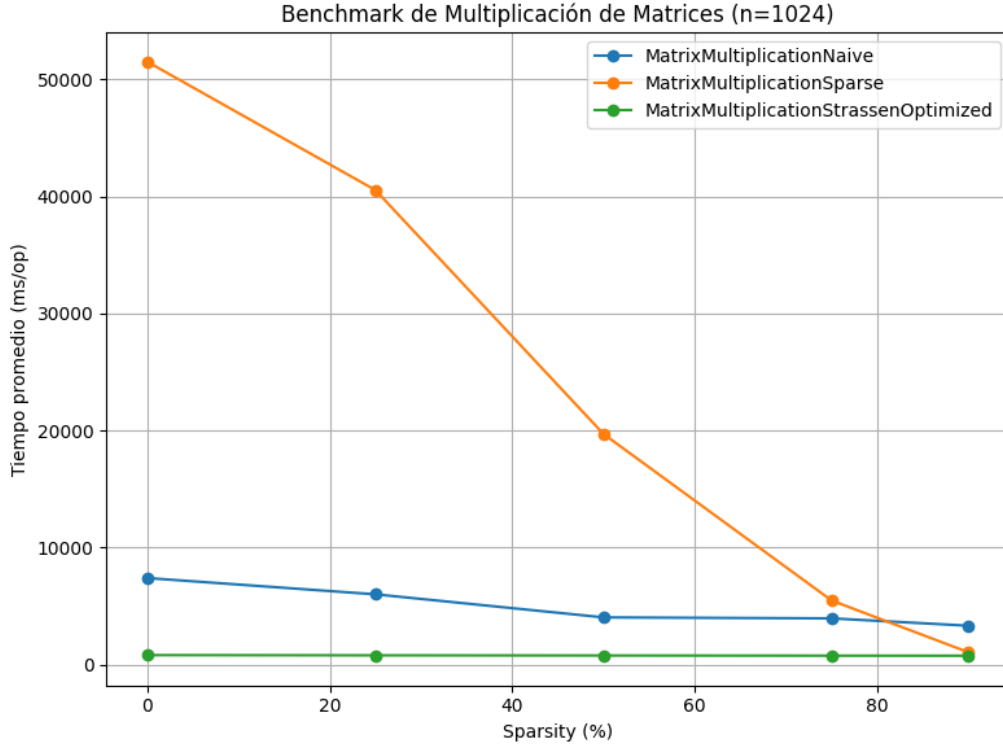


Figure 3: Performance comparison for $n = 1024$ across sparsity levels.

4 Discussion

- **Trade-offs:** Sparse multiplication excels only when sparsity is high; otherwise, overhead dominates.
- **Scalability:** StrassenOptimized provides a balanced approach, scaling better than naive multiplication while avoiding sparse overhead.
- **Practical Implications:** Algorithm choice should depend on matrix characteristics—dense vs. sparse, small vs. large.

5 Conclusion

This study demonstrates that:

- Naive multiplication is only viable for small matrices.
- Sparse multiplication is optimal for highly sparse matrices.
- StrassenOptimized offers robust performance across sizes and sparsity levels.

Future work includes extending benchmarks to parallel implementations, GPU acceleration, and hybrid algorithms combining dense and sparse strategies.

References

- Strassen, V. (1969). Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4), 354–356.
- Oracle. Java Microbenchmark Harness (JMH). <https://openjdk.org/projects/code-tools/jmh/>